WEB & API PROGRAMMING

Implementation Report

Topic 1: Reverse a String Program

TOPIC 1 — REVERSE A STRING PROGRAM

1. Prompt Used

Prompt Given:

Write a Python program to reverse a string entered by the user. Implement it using three methods: slicing, loop, and built-in reversed(). Also build a REST API endpoint using Flask that accepts a POST request with JSON { "text": "..." } and returns the reversed string.

2. Steps to Solve

1. Understand the problem — reverse every character in a given string
2. Identify suitable Python approaches (slicing, loop, built-in)
3. Write and test the console program with sample inputs
4. Wrap the logic in a Flask POST route /api/reverse
5. Validate inputs and handle edge cases (empty string, non-string)
6. Test the API using cURL or Postman and document outputs

3. Algorithm

Method 1 — Slicing ([::-1])

7. Accept string from user
8. Apply slice notation string[::-1] — step -1 reads from end to start
9. Return / print the result

Method 2 — For Loop

10. Initialize empty string reversed_str = ""
11. For each character c in input, prepend: reversed_str = c + reversed_str
12. After loop, reversed_str holds the reversed string

Method 3 — reversed() + join()

13. Pass string to Python's reversed() — returns a character iterator
14. Use "".join() to concatenate characters back into a string
15. Return the joined result

Method	Complexity & Notes
Slicing [::-1]	O(n) time · O(n) space · Fastest, most Pythonic
Loop (prepend)	O(n) time · O(n) space · Easy to understand
reversed() + join()	O(n) time · O(n) space · Clean & idiomatic

TOPIC 1 — SOURCE CODE

4. Python Console Program

```
# Reverse a String — Python Implementation
# Web & API Programming | 2025

def reverse_slicing(text):   return text[::-1]
def reverse_loop(text):
    r = ""
    for c in text: r = c + r
    return r
def reverse_builtin(text):   return ''.join(reversed(text))

if __name__ == '__main__':
    s = input("Enter a string: ")
    if not s.strip():
        print("Error: Input cannot be empty.")
    else:
        print(f"Slicing  : {reverse_slicing(s)}")
        print(f"Loop     : {reverse_loop(s)}")
        print(f"Built-in : {reverse_builtin(s)}")
```

5. Flask REST API — POST /api/reverse

```
from flask import Flask, request, jsonify

app = Flask(__name__)

@app.route('/api/reverse', methods=['POST'])
def reverse_string():
    data = request.get_json()
    if not data or 'text' not in data:
        return jsonify({'error': 'Missing field: text'}), 400
    original = data['text']
    if not isinstance(original, str):
        return jsonify({'error': 'text must be a string'}), 400
    return jsonify({
        'original' : original,
        'reversed' : original[::-1],
        'length'   : len(original),
        'status'   : 'success'
    }), 200

if __name__ == '__main__':
    app.run(debug=True, port=5000)
```

TOPIC 1 — DEBUGGING

6. Bug Log & Fixes

#	Bug	Cause	Fix
1	Empty string caused crash	No guard before processing	Added: if not s.strip() → error msg
2	API 500 on missing JSON body	get_json() returned None unchecked	Added: if not data or 'text' not in data
3	Non-string input: TypeError	No type check on incoming value	Added: isinstance(original, str) check
4	Unicode chars split wrong	Byte slicing broke multi-byte chars	Used str slice [::-1], not bytes slice
5	Browser CORS error on POST	Flask sends no CORS headers default	Added flask-cors: CORS(app) after init

7. Debug Trace — reverse_loop("Hello")

```
# Input: "Hello"   reversed_str = ""
# Step 1: c='H'  →  reversed_str = 'H'
# Step 2: c='e'  →  reversed_str = 'eH'
# Step 3: c='l'  →  reversed_str = 'leH'
# Step 4: c='l'  →  reversed_str = 'lleH'
# Step 5: c='o'  →  reversed_str = 'olleH'
# Final  : 'olleH'  ✓
```

8. API Debug — cURL Test

```
curl -X POST http://localhost:5000/api/reverse \
     -H 'Content-Type: application/json' \
     -d '{"text": "Hello, World!"}'

# Response 200 OK:
# { "original":"Hello, World!", "reversed":"!dlroW ,olleH",
#   "length":13, "status":"success" }
```

TOPIC 1 — OUTPUT

9. Console Program Output

```
Enter a string: Hello, World!
Slicing  : !dlroW ,olleH
Loop     : !dlroW ,olleH
Built-in : !dlroW ,olleH
```

```
Enter a string: racecar
Slicing  : racecar   ← Palindrome — same forwards and backwards
```

```
Enter a string:          ← (empty input)
Error: Input cannot be empty.
```

10. API Output — HTTP Responses

API Input	API Response
"Hello"	"olleH" — 200 OK
"12345"	"54321" — 200 OK
"racecar"	"racecar" — palindrome, 200 OK
"!@#$%"	"%$#@!" — 200 OK
{} (no text)	400 Bad Request: Missing field: text
text: 123 (int)	400 Bad Request: text must be a string

Result Summary:
All three methods (slicing, loop, reversed()) produce identical output for every test case. The Flask API correctly returns 200 OK for valid strings and 400 Bad Request for invalid inputs, with descriptive JSON error messages.

1. Prompt Used

Prompt Given:

Build a Login Authentication System using Python and Flask. Support: (1) POST /api/register — user registration with bcrypt-hashed passwords, (2) POST /api/login — login with JWT token generation, (3) GET /api/profile — protected route requiring a valid JWT Bearer token. Return proper HTTP status codes and JSON responses for all success and error cases.

2. Steps to Solve

16. Design the User model with fields: id, username, email, password (hashed), created_at
17. Set up Flask app with SQLAlchemy (SQLite) and flask-cors
18. Implement /api/register: validate input, check duplicates, hash password with bcrypt, save user
19. Implement /api/login: look up user, verify bcrypt hash, generate JWT token on success
20. Build token_required decorator to protect routes by decoding JWT from Authorization header
21. Implement /api/profile: return user data if token is valid
22. Test all routes, document bugs, and record outputs

3. Algorithm

Registration

23. Receive { username, email, password } via POST /api/register
24. Validate all fields present and password >= 8 characters
25. If username or email exists → return 409 Conflict
26. Hash password: bcrypt.hashpw(password.encode(), gensalt(12))
27. Save new User record → return 201 Created

Login & JWT Generation

28. Receive { username, password } via POST /api/login
29. Fetch user by username; if not found → 401 Unauthorized
30. Verify: bcrypt.checkpw(password, stored_hash) — if fails → 401
31. Generate JWT: payload = { user_id, exp: now + 24h } → sign with SECRET_KEY
32. Return 200 OK with { token, user_data }

Protected Route

33. Extract token from Authorization: Bearer <token> header
34. Decode JWT; catch ExpiredSignatureError → 401 / InvalidTokenError → 401
35. Fetch user from DB; if valid → return profile data with 200 OK

TOPIC 2 — SOURCE CODE (Part 1)

4. models.py — User Model

```
from flask_sqlalchemy import SQLAlchemy
from datetime import datetime
db = SQLAlchemy()

class User(db.Model):
    __tablename__ = "users"
    id         = db.Column(db.Integer, primary_key=True)
    username   = db.Column(db.String(80),  unique=True, nullable=False)
    email      = db.Column(db.String(120), unique=True, nullable=False)
    password   = db.Column(db.String(200), nullable=False)
    created_at = db.Column(db.DateTime, default=datetime.utcnow)
    is_active  = db.Column(db.Boolean,  default=True)

    def to_dict(self):
        return { 'id': self.id, 'username': self.username,
                 'email': self.email, 'is_active': self.is_active }
```

5. auth.py — JWT Helper & Decorator

```
import jwt
from datetime import datetime, timedelta, timezone
from functools import wraps
from flask import request, jsonify, current_app
from models import User

def generate_token(user_id):
    payload = { 'user_id': user_id,
                'exp': datetime.now(timezone.utc) + timedelta(hours=24) }
    return jwt.encode(payload, current_app.config['SECRET_KEY'], algorithm='HS256')

def token_required(f):
    @wraps(f)
    def decorated(*args, **kwargs):
        hdr = request.headers.get('Authorization', '')
        if not hdr.startswith('Bearer '):
            return jsonify({'error': 'Token missing'}), 401
        try:
            payload = jwt.decode(hdr.split()[1],
                current_app.config['SECRET_KEY'], algorithms=['HS256'])
            user = User.query.get(payload['user_id'])
            if not user: return jsonify({'error': 'User not found'}), 401
        except jwt.ExpiredSignatureError:
            return jsonify({'error': 'Token expired'}), 401
        except jwt.InvalidTokenError:
            return jsonify({'error': 'Invalid token'}), 401
        return f(user, *args, **kwargs)
    return decorated
```

TOPIC 2 — SOURCE CODE (Part 2 — Flask Routes)

6. app.py — Register, Login & Profile Routes

```
from flask import Flask, request, jsonify
from flask_cors import CORS
import bcrypt
from models import db, User
from auth import generate_token, token_required

app = Flask(__name__)
app.config['SECRET_KEY'] = 'change-in-production'
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///auth.db'
CORS(app); db.init_app(app)
with app.app_context(): db.create_all()

# ── REGISTER ──────────────────────────────────────
@app.route('/api/register', methods=['POST'])
def register():
    d = request.get_json() or {}
    u, e, p = d.get('username',''), d.get('email',''), d.get('password','')
    if not u or not e or not p:
        return jsonify({'error': 'All fields required'}), 400
    if len(p) < 8:
        return jsonify({'error': 'Password min 8 chars'}), 400
    if User.query.filter_by(username=u).first():
        return jsonify({'error': 'Username taken'}), 409
    hashed = bcrypt.hashpw(p.encode(), bcrypt.gensalt(12)).decode()
    db.session.add(User(username=u, email=e, password=hashed))
    db.session.commit()
    return jsonify({'message': 'Registered successfully'}), 201

# ── LOGIN ─────────────────────────────────────────
@app.route('/api/login', methods=['POST'])
def login():
    d = request.get_json() or {}
    u, p = d.get('username',''), d.get('password','')
    if not u or not p: return jsonify({'error': 'Fields required'}), 400
    user = User.query.filter_by(username=u).first()
    if not user or not bcrypt.checkpw(p.encode(), user.password.encode()):
        return jsonify({'error': 'Invalid credentials'}), 401
    return jsonify({'token': generate_token(user.id), 'user': user.to_dict()}), 200

# ── PROFILE (Protected) ───────────────────────────
@app.route('/api/profile', methods=['GET'])
@token_required
def profile(current_user):
    return jsonify({'user': current_user.to_dict()}), 200

if __name__ == '__main__': app.run(debug=True, port=5000)
```

TOPIC 2 — DEBUGGING

7. Bug Log & Fixes

#	Bug	Cause	Fix
1	500 on /register — db error	db.create_all() outside app context	Wrapped in: with app.app_context()
2	Password saved as bytes in DB	bcrypt returns bytes, stored raw	Added .decode('utf-8') before save
3	JWT decode TypeError in PyJWT v2	v2 returns str, not bytes	Removed .decode() from jwt.encode()
4	401 even with valid token	Header was 'Token' not 'Bearer'	Fixed: hdr.startswith('Bearer ')
5	IntegrityError on duplicate user	No pre-check before db.session.add()	Added filter_by().first() guard
6	CORS blocked from frontend JS	Flask missing CORS response headers	Applied: CORS(app) with flask-cors

8. JWT Debug Snippet

```
import jwt
SECRET = 'change-in-production'
token  = '<paste token from /api/login response>'

try:
    payload = jwt.decode(token, SECRET, algorithms=['HS256'])
    print('OK:', payload)
    # → {'user_id': 1, 'exp': 1720000000}
except jwt.ExpiredSignatureError: print('Token expired')
except jwt.InvalidTokenError as e: print('Invalid:', e)
```

9. bcrypt Verify Debug

```
import bcrypt
password = 'SecurePass123'
hashed   = bcrypt.hashpw(password.encode(), bcrypt.gensalt(12))
print(bcrypt.checkpw(password.encode(), hashed))       # True
print(bcrypt.checkpw(b'WrongPass',      hashed))       # False
```

TOPIC 2 — OUTPUT & SUMMARY

10. API Output — HTTP Responses

POST /api/register → 201 Created

```
Request : { "username":"john", "email":"j@x.com", "password":"Pass1234" }
Response: { "message": "Registered successfully" }
```

POST /api/login → 200 OK

```
Request : { "username":"john", "password":"Pass1234" }
Response: { "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6...",
            "user": { "id":1, "username":"john", "email":"j@x.com" } }
```

GET /api/profile → 200 OK (with valid Bearer token)

```
Header  : Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6...
Response: { "user": { "id":1, "username":"john", "email":"j@x.com" } }
```

GET /api/profile → 401 (expired / missing token)

```
Response: { "error": "Token expired" }
Response: { "error": "Token missing" }
```

11. Endpoint Summary

Endpoint	Description & Status Codes
POST /api/register	Register user with hashed password → 201 / 400 / 409
POST /api/login	Authenticate and return JWT token → 200 / 401
GET /api/profile	Return user profile (JWT required) → 200 / 401

Key Takeaways:

Topic 1 shows stateless REST API design with input validation. Topic 2 demonstrates industry-standard security: bcrypt password hashing, JWT-based stateless authentication, protected routes via decorators, and structured JSON error handling — all core skills in modern Web & API Programming.